Midterm Exam Review & Practice Problems

BUS 104: Decision Analysis and Management Science

Danko Turcic

UC RIVERSIDE | UNIVERSITY OF CALIFORNIA School of Business
A. GARY ANDERSON GRADUATE SCHOOL OF MANAGEMENT

Midterm Review Roadmap

1. Midterm Logistics, Topics & Resources
2. Python & PuLP 4.0 Practice
3. Solving LPs in Google Sheets & OpenSolver
4. Core LP & ILP Formulation Practice
5. Sensitivity Analysis Practice

Section 1

Midterm Logistics, Topics & Resources

Exam Structure

- **Online exam:** between 35 and 40 questions
- Availability and submission deadlines are on Canvas
- Exam is not timed
- Exam can be paused and resumed
- Two attempts available (grade based on the **last** attempt)
- If the average grade is less than 85%, the exam will be curved to an average of 85%
- Exam is open book, notes, etc.
- Okay to use AI
- Exam is an individual effort

Topics Covered on the Midterm

- Basic Python trivia questions
- Basic Spreadsheet trivia
- **Linear programming**
  - product mix
  - staffing
  - transportation
  - assignment
  - blending
- **Sensitivity analysis**
  - Reduced Costs
  - Shadow Prices
- **Integer linear programming**
  - 0-1 problems
  - fixed cost problems

Resources

- **Course AI Study Guide & Tutor (NotebookLM):**
  - Loaded with all course teaching materials to help you prepare for exams and homework:
  - `https://notebook.google.com/notebook/21540e36-100c-4abc-a851-af3bf66ef8e9`
- Practice problems posted on Canvas
- Homeworks!
- AI: utilize what is available, go crazy.

Section 2

Python & PuLP 4.0 Practice

Python Essentials 1: Lists, Indexing, and Slicing

Zero-Based Indexing and Negative Indices

```
# Zero-based indexing: index 0 is the 1st element
costs = [12, 25, 18, 40, 30]

first_item  = costs[0]    # 12
third_item  = costs[2]    # 18
last_item   = costs[-1]   # 30 (last element)
second_last = costs[-2]   # 40

# Slicing: list[start : stop] (stop is EXCLUDED)
sub_list  = costs[1:4]    # [25, 18, 40] (indices 1, 2, 3)
first_two = costs[:2]     # [12, 25]     (indices 0, 1)

# Built-in aggregations
n_items = len(costs)      # 5
total   = sum(costs)      # 125
```

Key Indexing Rules for the Exam

- **Zero-Indexed:** A list of length n has valid forward indices $0, 1, \ldots, n-1$.
- **Negative Indices:** `x[-1]` accesses the *last* element; `x[-2]` accesses the second-to-last element.
- **Half-Open Slices:** `x[a:b]` includes index `a` up to `b-1` (contains $b - a$ elements).

Common Exam Trap

`costs[1]` is the **second** element (25), *not* the first element (`costs[0] = 12`)!

Python Essentials 2: Dictionaries & List Comprehensions

Storing Parameters and Building Linear Expressions

```
products = ["Chairs", "Desks", "Tables"]
profit = {"Chairs": 40, "Desks": 70, "Tables": 55}
wood   = {"Chairs": 2,  "Desks": 4,  "Tables": 3}
qty    = {"Chairs": 10, "Desks": 5,  "Tables": 0}

# Dictionary key lookup & methods
desk_prof = profit["Desks"]          # 70
keys_list = list(profit.keys())      # ["Chairs", "Desks", "Tables"]
vals_list = list(profit.values())    # [40, 70, 55]

# List comprehension with sum() (dot product)
total_profit = sum(profit[p] * qty[p] for p in products)
# 40*10 + 70*5 + 55*0 = 750

# Filtering in a list comprehension
high_margin = [p for p in products if profit[p] >= 50]
# ["Desks", "Tables"]
```

Why Dictionaries Matter in PuLP

- Model parameters (c_i, a_{ij}, b_j) and decision variables (x_i) are indexed by product or node name using dictionaries.
- `profit["Desks"]` looks up the value associated with key `"Desks"`.
- `sum(c[i] * x[i] for i in items)` translates $\sum_i c_i x_i$ directly into Python!

Quick Check

What does `[2*k for k in [1, 2, 3]]` return?
Answer: `[2, 4, 6]`.

PuLP 4.0 Core Syntax Reference

The 4-Step Workflow for LP and MILP Models

```
import pulp

# 1. Initialize model (LpMaximize or LpMinimize)
prob = pulp.LpProblem("Model_Name", pulp.LpMaximize)

# 2. Declare variables (LpContinuous, LpInteger, LpBinary)
x = prob.add_variable("x", lowBound=0, cat=pulp.LpContinuous)
y = prob.add_variable("y", cat=pulp.LpBinary)

# Dictionary of variables across a list of items
items = ["A", "B"]
v = prob.add_variable_dicts("v", items, lowBound=0, cat=pulp.LpInteger)

# 3. Add objective (no inequality) & constraints (<=, >=, ==)
prob += 30 * x + 20 * v["A"] - 50 * y, "Objective"
prob += 2 * x + 1 * v["A"] <= 100, "Capacity_Limit"
prob += x - 40 * y <= 0, "Big_M_Switch"

# 4. Solve and inspect results
res = prob.solve()
print(res.status.name, pulp.value(prob.objective), x.varValue)
```

Critical PuLP 4.0 Rules

- **Sense:** `pulp.LpMaximize` or `pulp.LpMinimize`.
- **Variable Categories:**
  - `pulp.LpContinuous` (default)
  - `pulp.LpInteger` (general integer)
  - `pulp.LpBinary` (0-1 indicator)
- **Objective vs. Constraint:** Adding an expression *without* <=, >=, or == sets the objective; adding an expression *with* <=, >=, or == adds a constraint.
- **Values:** Use `x.varValue` for variable values and `pulp.value(prob.objective)` for $Z^{\star}$.

Python & PuLP Practice Questions (Part 1: Q1–Q3)

Self-Contained Exam-Style Trivia with Worked Solutions

Quick Syntax Reference: `list[0]` = 1st item | `list[-1]` = last item | `list[a:b]` = indices a to b-1 | `dict[key]` = value

Q1 (List Indexing & Slicing):
Given `demand = [50, 80, 120, 90, 60]`, what are the values of `demand[1]`, `demand[-1]`, and `sum(demand[1:3])`?

- **Answer:** `demand[1] = 80` (second element), `demand[-1] = 60` (last element), and `demand[1:3] = [80, 120]` so `sum(demand[1:3]) = 200`.

Q2 (Dictionary Comprehension):
Given `c = {"W1": 2, "W2": 4, "W3": 5}` and `x = {"W1": 50, "W2": 30, "W3": 0}`, what does `sum(c[w] * x[w] for w in c)` evaluate to?

- **Answer:**
  $2(50) + 4(30) + 5(0) = 100 + 120 + 0 = \mathbf{220}$.

Q3 (Conditional List Comprehension):
Given `hours = [15, 42, 38, 50, 20]`, what list is produced by `[h for h in hours if h >= 40]`?

- **Answer:** `[42, 50]` (filters only elements ≥ 40).

Exam Tip: Tracing Comprehensions

Always read `[expr for var in seq if cond]` in three steps:

1. Iterate `var` over `seq`.
2. Keep only items where `cond` is `True`.
3. Evaluate `expr` for each kept item.

Python & PuLP Practice Questions (Part 2: Q4–Q6)

Variable Types, Equality Constraints, and Shadow Price Extraction

PuLP 4.0 Reference: `cat=pulp.LpBinary` ($\{0,1\}$) | `==` for equality constraints | `c.pi` for constraint shadow price

Q4 (Variable Declaration):
How do you declare a binary decision variable $y \in \{0,1\}$ in `PuLP 4.0` vs. a non-negative integer variable $x \in \mathbb{Z}_{\geq 0}$?

- **Answer:**
```
y = prob.add_variable("y",
cat=pulp.LpBinary)
x = prob.add_variable("x", lowBound=0,
cat=pulp.LpInteger)
```

Q5 (Equality Constraint Syntax):
Why does `prob += x1 + x2 = 80` raise a Python `SyntaxError`, and how do you fix it?

- **Answer:** Single `=` is variable assignment in Python. Equality constraints require double equals: `prob += x1 + x2 == 80`.

Q6 (Retrieving Optimal Values & Duals):
After running `res = prob.solve()`, how do you retrieve (a) the optimal objective Z^*, (b) the optimal value of variable `xA`, and (c) the shadow price of each constraint?

- **Answer:**
  (a) `pulp.value(prob.objective)`
  (b) `xA.varValue` (or `pulp.value(xA)`)
  (c) `{c.name: c.pi for c in prob.constraints()}`

Watch Out for Default Bounds!

For `LpContinuous` and `LpInteger`, always specify `lowBound=0` if non-negativity ($x \geq 0$) is required!

Section 3

Solving LPs in Google Sheets & OpenSolver

Spreadsheet LP Layout & SUMPRODUCT Architecture

Translating Algebraic LPs into a Clean Cell Grid

Concrete 3-Product LP Grid (A1:F5):

Row	Col A	Col B	Col C	Col D	Col E (LHS)	Col F (RHS)
1	Product	Chairs	Desks	Tables	Total / Used	Limit
2	Units (x_j)	20	15	0		
3	Unit Profit	40	70	55	=SUMPRODUCT(B3:D3,B2:D2)	(Max Z)
4	Wood (bf)	2	4	3	=SUMPRODUCT(B4:D4,$B$2:$D$2)	<= 100
5	Labor (hr)	3	2	2.5	=SUMPRODUCT(B5:D5,$B$2:$D$2)	<= 90

- **Decision Variable Cells (B2:D2):** Highlighted in yellow; OpenSolver changes these values.
- **Objective Cell (E3):** =SUMPRODUCT(B3:D3, B2:D2) computes $40x_1 + 70x_2 + 55x_3$.

Why Absolute References $B$2:$D$2?

- Locking row 2 with dollar signs ($B$2:$D$2) in cell E4 allows you to drag the formula down to E5 without shifting the decision variable row!
- Omitting $ in =SUMPRODUCT(B4:D4, B2:D2) shifts B2:D2 to B3:D3 when dragged down to row 5 (multiplying labor by unit profit!).

Never Hardcode Numbers in LHS/Obj!

Cells E3:E5 must contain =SUMPRODUCT(...) formulas, while B2:D2 contain only numbers.

Configuring OpenSolver in Google Sheets

Step-by-Step Model Setup for LP and MILP Models

1. Core OpenSolver Dialog Fields

- **Objective Cell:** Select `E3` and choose **Max** (for profit) or **Min** (for cost).
- **Variable Cells:** Select the contiguous range `B2:D2`.
- **Constraints:**
  - Resource limits: `E4:E5 <= F4:F5`
  - Integer restriction: `B2:D2 int`
  - Binary (0-1) restriction: `B2:D2 bin`

2. Solver Engine & Non-Negativity

- **Unconstrained Variables Non-Negative:** Always **check** this box so $x_j \geq 0$ is enforced automatically.
- **Solving Method:** Select **CBC (Linear & Integer Solver)** / **Simplex LP**—never use a nonlinear engine for a linear program!

Checking Binding Constraints in Sheets

After solving, compare **Col E (LHS Used)** vs. **Col F (RHS Limit)**:

- LHS $=$ RHS $\implies$ **Binding** (Slack $= 0$, shadow price ≥ 0).
- LHS $<$ RHS $\implies$ **Non-binding** (Slack > 0, shadow price $= 0$).

Google Sheets & OpenSolver Practice Questions (Q1–Q4)

Self-Contained Spreadsheet Trivia with Worked Solutions

Grid Reference: `B2:D2` = Decision Variables (x_j) | `B3:D3` = Unit Profits (c_j) | `B4:D4` = Wood per unit | `E4` = Wood Used | `F4` = Wood Limit

Q1 (`SUMPRODUCT` Formula):
In the 3-product grid above, what exact formula goes into cell `E4` so it can be safely dragged down to `E5`?

- **Answer:** `=SUMPRODUCT(B4:D4, $B$2:$D$2)`.

Q2 (Relative Reference Bug):
Suppose a student enters `=SUMPRODUCT(B4:D4, B2:D2)` in `E4` and drags it down to `E5`. What formula appears in `E5`, and why is it wrong?

- **Answer:** `=SUMPRODUCT(B5:D5, B3:D3)`. It multiplies labor hours (`B5:D5`) by unit profits (`B3:D3`) instead of the decision variables (`B2:D2`)!

Q3 (OpenSolver Binary & Integer Setup):
How do you restrict cells `B2:D2` to be binary (0-1) variables in OpenSolver?

- **Answer:** Add a constraint selecting `B2:D2` on the left and choosing the `bin` operator from the relation dropdown.

Q4 (Identifying Slack & Shadow Price from the Grid):
After solving, cell `E4` (Wood Used) equals 100 (`F4 = 100`) while a Packaging row has `LHS = 65` and `RHS = 80`. Which resource has a zero shadow price?

- **Answer:** **Packaging** has $80 - 65 = 15$ units of leftover slack (non-binding), so its shadow price is strictly **$0**.

Section 4

Core LP & ILP Formulation Practice

Product Mix LP: Complete Worked Example

Allocating Scarce Wood and Labor Across Chairs and Desks

Problem Data:

- **Chairs (x_1):** Profit \$40/unit; uses 2 bf Wood and 3 hr Labor.
- **Desks (x_2):** Profit \$70/unit; uses 4 bf Wood and 2 hr Labor.
- **Available Resources:** 100 bf Wood; 90 hr Labor.

LP Formulation:

$$\max \; Z = 40x_1 + 70x_2$$

$$\text{s.t.} \quad \begin{cases} 2x_1 + 4x_2 \le 100 & \text{(Wood)} \\ 3x_1 + 2x_2 \le 90 & \text{(Labor)} \\ x_1, x_2 \ge 0 \end{cases}$$

Corner-Point Solution & Optimal Mix

Solve the intersection of the two binding constraints:

$$\begin{cases} 2x_1 + 4x_2 = 100 \implies x_1 + 2x_2 = 50 \\ 3x_1 + 2x_2 = 90 \end{cases}$$

Subtracting Eq. (1) from Eq. (2) gives:

$$2x_1 = 40 \implies \mathbf{x_1^\star = 20} \text{ Chairs}, \quad \mathbf{x_2^\star = 15} \text{ Desks}.$$

Maximum Profit:

$Z^\star = 40(20) + 70(15) = 800 + 1050 = \mathbf{\$1,850}$.

Corner Point Comparison

$(0,0) \to \$0$ | $(30,0) \to \$1,200$ | $(0,25) \to \$1,750$ | $\mathbf{(20,15)} \to \mathbf{\$1,850}^\star$.

Staffing Problem: Description

Problem Context:

- A call center must schedule workers to cover daily shifts.
- Each shift requires a minimum number of employees.
- Employees work 5 consecutive days followed by 2 days off.
- The objective is to minimize total staffing costs while meeting coverage.

Goal:

- Decide how many employees to start on each day of the week.
- Ensure daily staffing requirements are satisfied at minimum cost.

Staffing Problem: Decision Variables and Constraints

Decision Variables:

- x_i = number of employees who start their 5-day workweek on day i, $i = 1, \ldots, 7$

Parameters:

- c_i = cost of assigning an employee starting on day i
- r_j = minimum required employees on day j

Key Constraints:

- Each day must have at least r_j employees working.
- Employees work 5 consecutive days, off 2 days.

Staffing Problem: LP Formulation

$$\min Z = \sum_{i=1}^{7} c_i x_i$$

$$\text{s.t.} \begin{cases} x_1 + x_4 + x_5 + x_6 + x_7 \geq r_1 & \text{(Mon)} \\ x_1 + x_2 + x_5 + x_6 + x_7 \geq r_2 & \text{(Tue)} \\ x_1 + x_2 + x_3 + x_6 + x_7 \geq r_3 & \text{(Wed)} \\ x_1 + x_2 + x_3 + x_4 + x_7 \geq r_4 & \text{(Thu)} \\ x_1 + x_2 + x_3 + x_4 + x_5 \geq r_5 & \text{(Fri)} \\ x_2 + x_3 + x_4 + x_5 + x_6 \geq r_6 & \text{(Sat)} \\ x_3 + x_4 + x_5 + x_6 + x_7 \geq r_7 & \text{(Sun)} \\ x_i \geq 0 \quad \forall i \end{cases}$$

Blending Problem: Description

Problem Context:

- A refinery produces a fuel blend using two raw materials.
- Each raw material has a different cost and quality content.
- The final product must meet a minimum quality standard.
- The goal is to minimize total cost while satisfying demand and quality.

Blending Problem: Decision Variables and Constraints

Decision Variables:

- x_1 = barrels of crude A used
- x_2 = barrels of crude B used

Parameters:

- c_1, c_2 = costs per barrel of A and B
- q_1, q_2 = quality measures of A and B
- Q_{min} = minimum required blend quality
- D = demand (barrels of finished blend)

Key Constraints:

- $x_1 + x_2 = D$ (total demand)
- Weighted average quality $\geq Q_{\text{min}}$
- $x_1, x_2 \geq 0$

Blending Problem: LP Formulation

$$\min Z = c_1 x_1 + c_2 x_2$$

$$\text{s.t.} \begin{cases} x_1 + x_2 = D \\ q_1 x_1 + q_2 x_2 \geq Q_{\min}(x_1 + x_2) \\ x_1, x_2 \geq 0 \end{cases}$$

Interpretation: Choose crude mix to meet quality and demand at lowest cost.

Transshipment Problem: Description

Problem Context:

- A company ships goods from factories to warehouses, then to retailers.
- Each arc (route) has a cost per unit shipped.
- Factory supplies, warehouse capacities, and retailer demands are known.
- The objective is to minimize total shipping cost.

Transshipment Problem: Decision Variables and Constraints

Decision Variables:

- x_{ij} = units shipped from node i to node j

Parameters:

- c_{ij} = shipping cost per unit from i to j
- s_i = supply at source i
- d_j = demand at destination j

Key Constraints:

- Flow conservation at each node:

$$\sum_j x_{ij} - \sum_k x_{ki} = b_i$$

where $b_i > 0$ for sources, $b_i < 0$ for sinks, $b_i = 0$ for transshipment nodes.

Transshipment Problem: LP Formulation

$$\min Z = \sum_i \sum_j c_{ij} x_{ij}$$

$$\text{s.t.} \quad \sum_j x_{ij} - \sum_k x_{ki} = b_i \quad \forall i$$

$$x_{ij} \geq 0 \quad \forall i, j$$

Interpretation: Ship goods through a network of factories, warehouses, and retailers to minimize total transport cost.

Assignment Problem: Description

Problem Context:

- A manager must assign a set of workers to a set of tasks.
- Each worker can perform only one task, and each task requires exactly one worker.
- The cost (or time) of assigning worker i to task j is known.
- The objective is to minimize the total assignment cost.

Goal:

- Determine the optimal one-to-one assignment between workers and tasks that yields the lowest total cost.

Assignment Problem: Decision Variables and Constraints

Decision Variables:

- $x_{ij} = \begin{cases} 1 & \text{if worker } i \text{ is assigned to task } j \\ 0 & \text{otherwise} \end{cases}$

Parameters:

- c_{ij} = cost (or time) of assigning worker i to task j

Key Constraints:

- Each worker is assigned to exactly one task:

$$\sum_j x_{ij} = 1 \quad \forall i$$

- Each task receives exactly one worker:

$$\sum_i x_{ij} = 1 \quad \forall j$$

- Binary restriction: $x_{ij} \in \{0, 1\}$

Assignment Problem: Full LP Formulation

$$\min Z = \sum_i \sum_j c_{ij} x_{ij}$$

$$\text{s.t.} \begin{cases} \sum_j x_{ij} = 1 & \forall i \quad \text{(each worker does one task)} \\ \sum_i x_{ij} = 1 & \forall j \quad \text{(each task has one worker)} \\ x_{ij} \in \{0, 1\} & \forall i, j \end{cases}$$

Interpretation: Find the assignment pattern minimizing total cost (or maximizing efficiency) under one-to-one matching constraints.

Integer Linear Programming (ILP) Formulation Cheat Sheet

0-1 Logical Conditions, Fixed Setup Costs, and Minimum Order Quantities

1. Logical Conditions on Binary Variables ($y_i \in \{0, 1\}$)

- **At most one (Mutual Exclusivity):**

$$y_1 + y_2 \leq 1$$

- **Exactly k out of n projects:**

$$\sum_{i=1}^{n} y_i = k$$

- **Contingency (Project 2 requires Project 1):**

$$y_2 \leq y_1 \iff y_2 - y_1 \leq 0$$

- **Complementary (Both or Neither):**

$$y_1 = y_2 \iff y_1 - y_2 = 0$$

2. Fixed Setup Charge & Big-M Linking

If producing $x > 0$ incurs fixed fee F, introduce binary $y \in \{0, 1\}$:

- **Objective:** Subtract Fy (max profit) or add Fy (min cost).
- **Big-M Linking Constraint:**

$$x \leq My \iff x - My \leq 0$$

where M is the tightest natural upper bound on x.

3. Minimum Order Quantity (MOQ L)

To force $x = 0$ (if $y = 0$) or $L \leq x \leq M$ (if $y = 1$):

$$Ly \leq x \leq My \iff x - Ly \geq 0 \text{ and } x - My \leq 0$$

Section 5

Sensitivity Analysis Practice

New Product Decision: Problem Description

Context:

- A firm produces Products A and B using two scarce resources: Machine Hours (M) and Labor Hours (L).
- A new Product C is being considered.
- We can use **shadow prices** and **reduced cost** from the current LP solution to decide if C should be added.

Goal:

- Determine whether introducing Product C would increase total profit.

Base Linear Program (Current Products A and B)

Current LP:

$$\max Z = 30x_A + 20x_B$$

$$\text{s.t.} \begin{cases} 2x_A + 1x_B \leq 100 & \text{(Machine hours M)} \\ 1x_A + 1.5x_B \leq 90 & \text{(Labor hours L)} \\ x_A, x_B \geq 0 \end{cases}$$

Optimal Primal Solution:

$$x_A^\star = 30, \quad x_B^\star = 40, \quad Z^\star = 30(30) + 20(40) = \$\mathbf{1700}$$

Unique Shadow Prices (from binding constraints M and L):

$$\begin{cases} 2y_M^\star + 1y_L^\star = 30 \\ 1y_M^\star + 1.5y_L^\star = 20 \end{cases} \implies \mathbf{y_M^\star = \$12.50}, \quad \mathbf{y_L^\star = \$5.00}$$

Check Dual Objective: $100(12.50) + 90(5.00) = 1250 + 450 = \$1700 = Z^\star$

Candidate Product C

Product C Data:

	Profit c_i	Resource Use (M, L)
Product C	\$26	(1, 2)

Idea:

- Product C earns \$26 per unit and uses 1 hour of Machine time and 2 hours of Labor.
- We test profitability using the *reduced cost* (net marginal benefit) formula:

$$r_C = c_C - \sum_i y_i^\star a_{iC} = c_C - (y_M^\star a_{MC} + y_L^\star a_{LC})$$

Reduced Cost Calculation and Decision

Compute Reduced Cost for Product C:

$$r_C = 26 - (12.50 \times 1 + 5.00 \times 2) = 26 - (12.50 + 10.00) = \boxed{+\$3.50 > 0}$$

Interpretation:

- $r_C = +\$3.50 > 0 \implies$ Each unit of Product C adds \$3.50 in net profit above the opportunity cost of its resources (\$22.50).
- Therefore, **Product C should be produced** (it enters the optimal basis).

Example of a Non-Profitable Candidate (Product D, $c_D = \$18$, uses 2 M, 1 L):

$$r_D = 18 - (12.50 \times 2 + 5.00 \times 1) = 18 - 30.00 = -\$12.00 \leq 0$$

Product D should not be added (opportunity cost of resources is \$30, exceeding its \$18 profit).

Interpreting Shadow Prices

Shadow Price ($y_i^\star$):

- Represents the change in optimal objective value $Z^\star$ per unit increase in the RHS of a binding constraint.
- Example (Machine Hours):

$$\frac{\Delta Z^\star}{\Delta \text{Machine hours}} = y_M^\star = \$12.50$$

- Meaning: one extra machine hour increases maximum profit by **$12.50**.

Valid Range:

- Shadow prices remain constant only within the allowable RHS range where the current optimal basis remains feasible ($60 \leq \text{RHS}_M \leq 180$).

Effect of RHS (Resource) Changes

Scenario: Increase Machine Hours by 5 units ($\mathbf{RHS}_M : 100 \to 105$)

$$\Delta Z = y_M^\star \times 5 = 12.50 \times 5 = \$62.50$$

$$Z_{\text{new}} = 1700 + 62.50 = \boxed{\$1762.50}$$

Interpretation:

- Within the allowable range, each additional 1 hour of M raises profit by \$12.50.
- If M increases beyond 180 hours, Labor becomes the sole bottleneck, the basis changes, and $y_M^\star = \$12.50$ no longer holds.

Takeaway: Shadow prices and reduced costs help managers evaluate new products or resource changes *without re-solving the LP from scratch*.

Verifying Shadow Prices & Reduced Costs in Python (PuLP 4.0)

Extracting Dual Prices (c.pi) and Testing Candidate Products

```
import pulp

prob = pulp.LpProblem("Base_Product_Mix", pulp.LpMaximize)
xA = prob.add_variable("xA", lowBound=0, cat=pulp.LpContinuous)
xB = prob.add_variable("xB", lowBound=0, cat=pulp.LpContinuous)

prob += 30 * xA + 20 * xB, "Total_Profit"
prob += 2 * xA + 1 * xB <= 100, "Machine_Hours"
prob += 1 * xA + 1.5 * xB <= 90, "Labor_Hours"

res = prob.solve()
z_opt = pulp.value(prob.objective)
print(f"Status: {res.status.name}, Z* = ${z_opt:.2f}")
print(f"xA* = {xA.varValue:.1f}, xB* = {xB.varValue:.1f}")

# Extract Shadow Prices from prob.constraints() list in PuLP 4.0
duals = {c.name: abs(c.pi) for c in prob.constraints()}
print("Shadow Prices:", duals)
r_C = 26 - (1 * duals["Machine_Hours"] + 2 * duals["Labor_Hours"])
print(f"Reduced Cost of Product C: r_C = +${r_C:.2f}")
```

Console Output

```
Status:  Optimal, Z* = $1700.00
xA* = 30.0, xB* = 40.0
Shadow Prices:
{'Machine_Hours':  12.5,
 'Labor_Hours':  5.0}
Reduced Cost of Product C:
r_C = +$3.50
```

PuLP 4.0 Note

In PuLP 4.0, `prob.constraints()` is a method returning a list of constraint objects. Access `c.name` and `abs(c.pi)` to retrieve shadow prices!